Programmation fonctionnelle (OCaml)

1. Types et opérateurs

Type	Valeurs	Opérations
int	3, −7, 0	+ − * / (division entière) mod
float	3.0, 3., 2.5	**+. −. *. /.** sqrt
bool	true, false	&& || not
string	"bonjour"	^ (concaténation)

- Comparaisons = <> < > <= >= entre deux valeurs **du même type** → bool (marche aussi sur les string : ordre alphabétique).
- **Jamais de mélange int/float** : float_of_int 3 = 3. ; int_of_float 2.9 = 2 (tronque).
- L'application d'une fonction passe avant tout : `f n - 1` = (f n) − 1.

7 / 2 = 3 (int) · 7. /. 2. = 3.5 · 7 mod 2 = 1 · 7.2 / 2 **erreur** · 3 * 4. **erreur** · 7. mod 2. **erreur** · int_of_float(10.0 /. 2.5) = 4 · int_of_float(10.0) / int_of_float(2.5) = 5 · not (true) && not (false) = false · "bon" ^ "jour" = "bonjour" · not("true") **erreur** · true = "true" **erreur**.

Partiel 2021 Q1 : List.hd [4; 2] + 1 → int, 5 · (8/3) :: (8 mod 3) :: [] → int list, [2; 2] · List.hd ["un"; "deux"] < "trois" → bool, false (u après t).

2. if, let … in, let … and

- **if c then e1 else e2** : c bool, **e1 et e2 du même type** (le type du if). Pas de else : seulement si type unit.
- **let x = e1 in e2** : x n'existe que dans e2. **let a = … and b = …** : simultanés, b ne voit pas a.

if 1 < 2 then 3+1 else 5/2 → int, 4 · if 1 < 3/2 then … else 3 → 3 (3/2 = 1) · if 3/2 < 2 then 4.1 **erreur** (pas de else) · if 3/2 < 2 then 4 < 5 else 4 + 5 **erreur** (bool / int).
TD 3 : let x = 5 in x*x → 25 · let a = 2 and b = true in b && (a > 0) → true · let a = 2 and b = 5 in a + 2*b → 12 · let a = 5 and b = a+1 in … **erreur** · (a < b) && ((a > b)/2) **erreur** · let a = 1 and b = 1 and c = 2 in a = b = c **erreur** : (a = b) = c compare true à 2.

3. Fonctions et signatures

```
let f : int -> int = function n -> 2 * n + 1
let g : int * bool -> int =
  function (x, y) -> if y then x + 1 else 2 * x
let h : int -> int -> int = fun a b -> a + b
(* appels : f 3   g (2, true)   h 2 3   h 2 : int -> int *)
```

- **Couple** : function (x, y) → type `t1 * t2 -> t`, appel f (a, b). **Curryfiée** : fun x y → `t1 -> t2 -> t`, appel f a b. Plusieurs arguments : **curryfiée** (consigne d'examen).
- Trouver une signature : ce que **chaque opérateur impose** (x + 1 → int ; if y → bool ; +. → float ; List.hd y → liste ; y :: … → élément).

TD 7 : f1 = function (x, y) -> if y then x+1 else 2*x : **int * bool -> int** · f2 = function (x, y) -> if x > y then x > y else y > 2*x : **int * int -> bool** · Partiel : h = function (x, y) -> x + 2 * List.hd(y) : **int * int list -> int** · memeParite = function (a, b) -> a mod 2 = b mod 2 : int * int -> bool.

4. Récursivité sur les entiers

1. cas de base (n = 0, n < 10, i = 0) · **2.** récurrence : f n avec f (n − 1) · **3. let rec**, et **des parenthèses** : f (n - 1). Base : **0** pour une somme ou un compte, **1** pour un produit, **true** pour « tous ».

```
let rec som : int -> int = function
  n -> if n = 0 then 0 else n + som (n - 1)
let rec f : int -> int = function
  n -> if n = 0 then 5 else 2 * n + f (n - 1)
let rec mul : int * int -> int = function
  (a, b) -> if a = 0 then 0 else b + mul (a - 1, b)
let rec carre : int -> int = function
  n -> if n = 0 then 0 else carre (n - 1) + n + n - 1
let rec estPuis10 : int -> bool = function
  n -> n = 1 || (n > 0 && n mod 10 = 0 && estPuis10 (n / 10))
let rec queDesUn : int -> bool = function
  n -> if n < 10 then n = 1
       else n mod 10 = 1 && queDesUn (n / 10)
let rec prodDiv : int -> int -> int = fun n i ->
  if i = 0 then 1
  else if n mod i = 0 then i * prodDiv n (i - 1)
  else prodDiv n (i - 1)
let rec nbDiv : int -> int -> int = fun n i ->
  if i = 0 then 0
  else (if n mod i = 0 then 1 else 0) + nbDiv n (i - 1)
let estPremier : int -> bool = function n -> nbDiv n n = 2
```

TD 12 : f n = n(n + 1) + 5 · queDesUn : partiel Q3 · prodDiv : partiel Q4 · n mod 10 : dernier chiffre · n / 10 : n sans son dernier chiffre · prodDiv 45 7 = 15 · prodDiv 12 5 = 24 · nbrep (n, b) = nbrep (n, b−1) + nbrep (n−1, b) ; b = 0 → (1 si n = 0 sinon 0) ; n = 0 → 1.

5. Les listes

- `[2; 3; 2; 4]` avec des **points-virgules** ; [] vide. `[2, 3]` = liste à **un** élément, le couple (2, 3).
- `x :: li` ajoute devant (à droite : une liste) · `li1 @ li2` colle deux listes · `List.hd li` premier · `List.tl li` le reste. Sur [] : erreur.
- `List.tl li = []` : la liste a un seul élément.

TD 22, li = [2; 3; 2; 4] : List.tl (List.tl li) = [2; 4] · List.hd (List.tl (List.tl li)) = 2 · List.hd de 4 List.tl **erreur** (liste vide) · 7 :: li = [7; 2; 3; 2; 4] · 7 :: List.tl li = [7; 3; 2; 4] · List.hd li :: List.tl li = li · List.hd (List.tl li) :: List.tl li = [3; 3; 2; 4] · List.hd ["liste"; "entiers"] :: ["chaines"] = ["liste"; "chaines"] · 8 :: 3 **erreur** · 8 :: [3] = [8; 3].

6. Le schéma récursif sur une liste

```
let rec g : int list -> … = function
  li -> if li = [] then (valeur de base)
        else … List.hd li … g (List.tl li) …
```

L'énoncé demande	base	cas général
somme, longueur, compte	0	… + g (List.tl li)
produit	1	… * g (List.tl li)
tous vérifient	true	(test sur List.hd li) && g (List.tl li)
il en existe un	false	(test) || g (List.tl li)
garder certains	[]	if test then List.hd li :: g (List.tl li) else g (List.tl li)
transformer chacun	[]	(calcul sur List.hd li) :: g (List.tl li)
liste non vide	if List.tl li = [] then List.hd li else … g (List.tl li) …	

7. Partiel 2021 (Q5, Q6) et cours

```
let rec somElem : int list -> int = function
  li -> if li = [] then 0
        else List.hd li + somElem (List.tl li)
let rec prefLong : int list -> int -> int list =
  fun li n ->
    if li = [] || n = 0 then []
    else List.hd li :: prefLong (List.tl li) (n - 1)
let rec longueur : int list -> int = function
  li -> if li = [] then 0
        else 1 + longueur (List.tl li)
let rec ajout1 : int list -> int list = function
  li -> if li = [] then []
        else (List.hd li + 1) :: ajout1 (List.tl li)
```

8. TD 23 à 26

```
let inverserElem : int list -> int list = function
  li -> List.hd (List.tl li) :: List.hd li
        :: List.tl (List.tl li)
let rec listeEntiers : int -> int list = function
  n -> if n = 0 then [0]
       else n :: listeEntiers (n - 1)
let rec derListe : int list -> int = function
  li -> if List.tl li = [] then List.hd li
        else derListe (List.tl li)
let rec maxListe : int list -> int = function
  li -> if List.tl li = [] then List.hd li
        else max (List.hd li) (maxListe (List.tl li))
```

9. TD 27 et 28 : tous, il existe, garder, enlever

```
let rec tousPairs : int list -> bool = function
  li -> if li = [] then true
        else List.hd li mod 2 = 0 && tousPairs (List.tl li)
let rec exiPairs : int list -> bool = function
  li -> if li = [] then false
        else List.hd li mod 2 = 0 || exiPairs (List.tl li)
let rec liPairs : int list -> int list = function
  li -> if li = [] then []
        else if List.hd li mod 2 = 0
        then List.hd li :: liPairs (List.tl li)
        else liPairs (List.tl li)
let rec oterPairs : int list -> int list = function
  li -> if li = [] then []
        else if List.hd li mod 2 = 0
        then oterPairs (List.tl li)
        else List.hd li :: oterPairs (List.tl li)
let rec liDivInf : int -> int -> int list =
  fun n i ->
    if i = 0 then []
    else if n mod i = 0 then i :: liDivInf n (i - 1)
    else liDivInf n (i - 1)
let liDiv : int -> int list = function n -> liDivInf n n
```

10. TD 29 à 31, et nbOcc (35)

```
let rec ajouFin : int -> int list -> int list =
  fun x li -> if li = [] then [x]
              else List.hd li :: ajouFin x (List.tl li)
let rec inverseLi : int list -> int list = function
  li -> if li = [] then []
        else ajouFin (List.hd li) (inverseLi (List.tl li))
let rec prefixe : int list -> int list -> bool =
  fun l1 l2 -> if l1 = [] then true
               else if l2 = [] then false
               else List.hd l1 = List.hd l2
                    && prefixe (List.tl l1) (List.tl l2)
let rec suffixe : int list -> int list -> bool =
  fun l1 l2 -> l1 = l2
               || (l2 <> [] && suffixe l1 (List.tl l2))
let rec facteur : int list -> int list -> bool =
  fun l1 l2 -> prefixe l1 l2
               || (l2 <> [] && facteur l1 (List.tl l2))
let rec nbOcc : int -> int list -> int =
  fun x li ->
    if li = [] then 0
    else (if List.hd li = x then 1 else 0) + nbOcc x (List.tl
li)
```

11. Au cas où : match

`match e with | motif -> … | motif when cond -> … | _ -> …` : le **premier** motif qui correspond, de haut en bas. Sur une liste : `| [] -> … | x :: r -> …` (x = List.hd, r = List.tl). TD 32 : n = 0 → 0 · n = 6 → 3 · n = 3 → 4 (le cas « 3 » n'est jamais atteint).

Erreurs classiques · avant de rendre

- ☐ **rec** si la fonction s'appelle elle-même.
- ☐ Parenthèses : f (n - 1), somElem (List.tl li), jamais somElem List.tl li.
- ☐ Signature écrite (int list, pas int) ; curryfiée dès 2 arguments.
- ☐ Un else à chaque if, le même type dans les deux branches.
- ☐ Pas de List.hd sans avoir testé li = [] (ou List.tl li = []).
- ☐ +. *. /. pour les float ; :: pour un élément, @ pour deux listes.
- ☐ Vérifier sur l'exemple de l'énoncé en déroulant : somElem [6; 9; 5] = 6 + (9 + (5 + 0)) = 20.